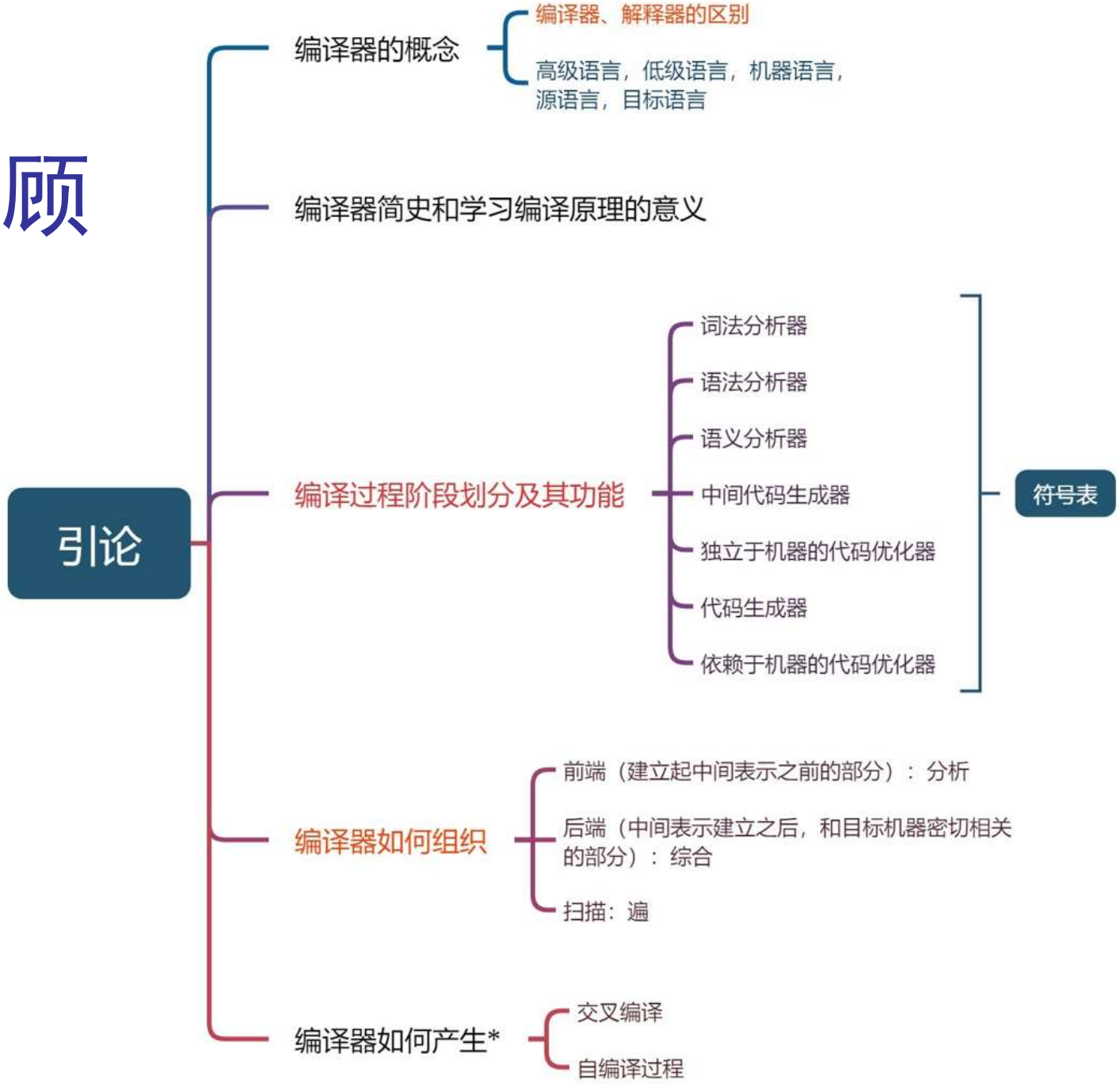


编译原理

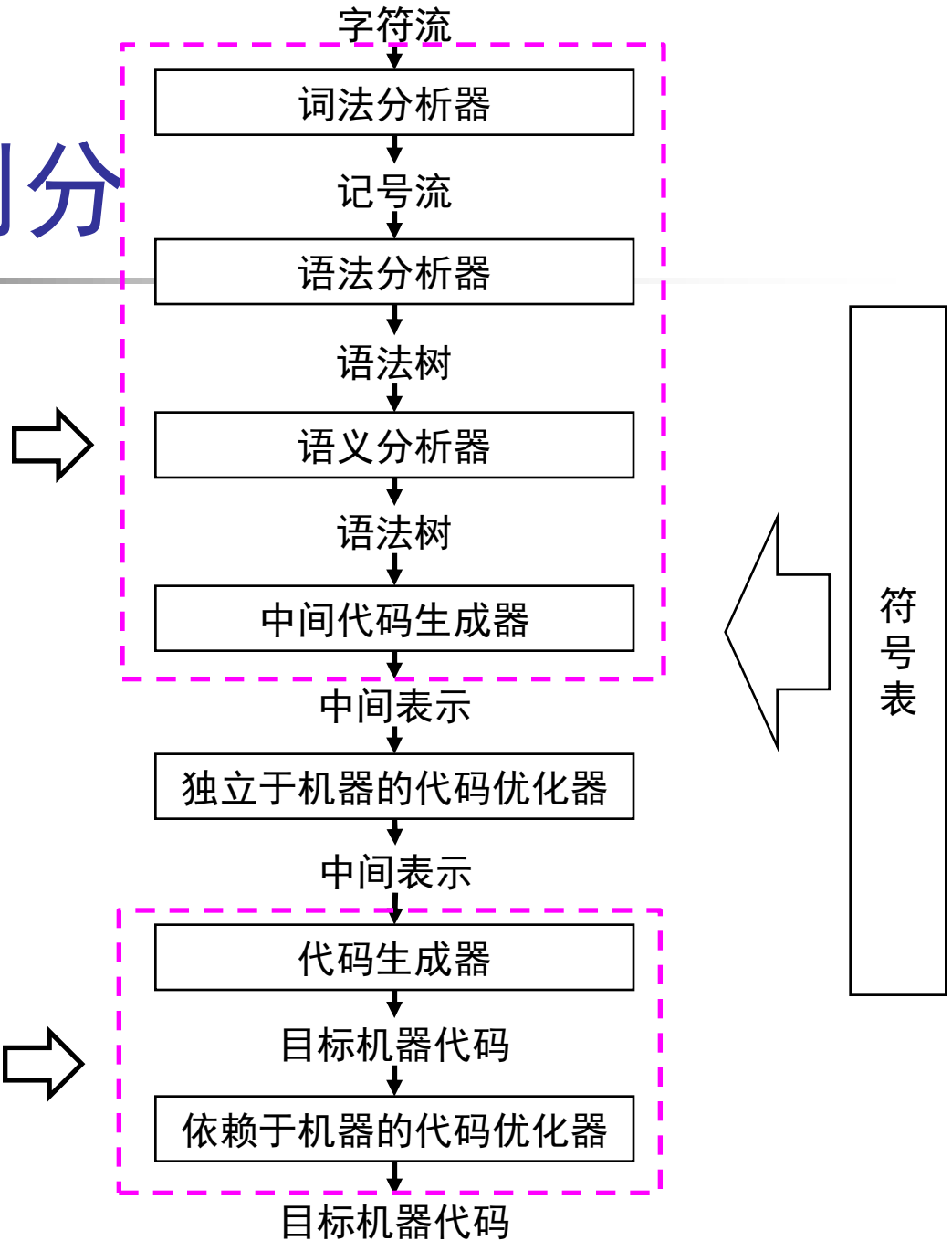
中国海洋大学计算机系
葛 琳
gelin@ouc.edu.cn

回顾

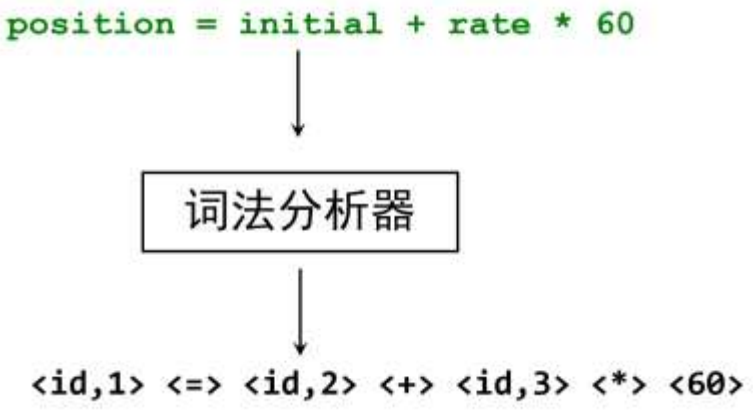

引论
编译器的概念
编译器、解释器的区别
高级语言，低级语言，机器语言，源语言，目标语言
编译器简史和学习编译原理的意义
编译过程阶段划分及其功能
词法分析器
语法分析器
语义分析器
中间代码生成器
独立于机器的代码优化器
代码生成器
依赖于机器的代码优化器
符号表
编译器如何组织
前端（建立起中间表示之前的部分）：分析
后端（中间表示建立之后，和目标机器密切相关的部分）：综合
扫描：遍
编译器如何产生*
交叉编译
自编译过程

回顾：编译器的划分

- 前端（front-end）：分析部分，与源语言密切相关。
- 后端（back-end）：综合部分，与目标机器更相关



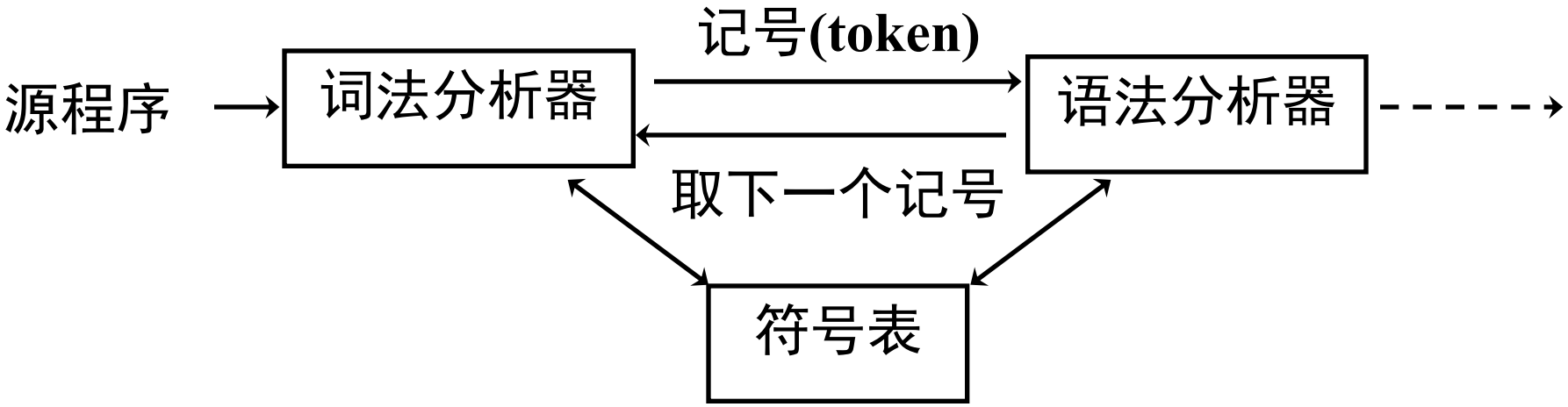
第二章 词法分析



- 词法分析的主要功能：
  - 源程序字符流（characters）→ 词法记号流（tokens）
- 手工构造词法分析器的方法：
  - 用状态转换图描述源语言词法记号的结构
  - 手工将状态转换图翻译成识别词法记号的程序
- 词法分析器的自动生成：正规式、状态转换图和有限自动机等。

第二章 词法分析（续）

- 词法分析器和语法分析器的一种典型关系：语法分析器为主导



第二章 词法分析（续）

- 词法分析器的任务：
  - 将正确的词法单元/单词（lexeme）识别为记号；
  - 剥去注释和空白（制表符、回车符、空格等分隔符）
  - 将各阶段产生的错误信息和源程序联系起来
  - 如果源语言支持宏，则宏的预处理可以在词法分析时实现

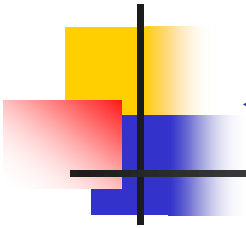
词法分析功能示例：

```
if (sum > 0)
  return sum;
```

```
\tif (sum > 0)\n\t\treturn sum;
```

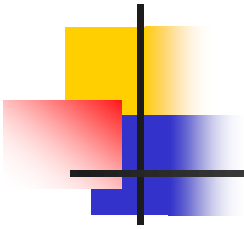
```
  K  ( ID  > NUM )       K        ID ;
```

token class: WS, ID, Keywords(K), NUM……



本次课内容

- 词法记号及其属性
- 词法错误
- 词法记号的描述与识别：
  - 串和语言
  - 正规式
  - 正规定义
  - 状态转换图



2.1 词法记号及属性

- 词法记号（tokens）、模式（patterns）、词法单元/单词（lexemes）
- 词法记号的属性
- 词法错误

2.1.1 词法记号、模式、词法单元

- 词法记号（token，简称记号）：
  - 计算机内部表示一般为<记号名，属性值> 二元组，属性值为可选的。
  - 记号名即记号分类（token class），是语法分析的输入符号，记号名常用来直接引用记号。
- 模式（pattern）：记号的模式描述属于该记号的词法单元的形式。（一种模式可能有多个字符串可以匹配它）
- 词法单元/单词（lexeme）：源程序中匹配一个记号模式的字符序列，它是该记号的实例

2.1.1 词法记号、模式、词法单元（续）

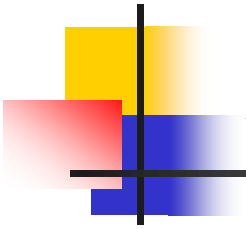
- 记号、模式和词法单元/单词的例子：

词法记号（名）	词法单元/单词 例举	模式的非形式描述
if	`if`	字符i, f
for	`for`	字符 f, o, r
relation	`<, <=, =, …`	< 或 <= 或 = 或 …
id	`sum, count, D5`	由字母开头的字母数字串
num	`3.1, 10, 2.8E12`	任何数值常数
literal	`“seg.error”`	引号“和”之间的任意字符串，但引号本身除外

2.1.1 词法记号、模式、词法单元（续）

- 如何确定记号名（即token class）？
  - 确定记号名（即记号分类）没有原则性规定，取决于处理上的方便。
  - 通常关键字和分隔符等可以一词一类；
  - 标识符通常算做一类；
  - 常数或常量可按数据类型分类，如前例中的num，literal。
  - 运算符可以一词一类，也可以若干个归为一类，如前例中的relation。

2.1.1 词法记号、模式、词法单元（续）



- 试着人工将下列源程序片段转换成记号流（可以只用记号名表示记号，记号名的选择按上页描述的原则）：
  - 源程序片段：（其中**sum**是变量）

```
if (sum > 0) return sum;
```

IF LPARE ID GT NUM RPARE

RETURN ID SEMICOMMA

2.1.1 词法记号、模式、词法单元（续）

- 练习：请将下面的C语句字符序列转换为记号流（记号名可以按前述规则自定义，属性值可省略或用lexeme表示，即记号二元组表示为<token class, lexeme>）

```
printf(“Total=%d\n”, score);//ignore
```

<ID,printf> <LPARE>
<LITERAL,”Total=%d\n”> <COMMA>
<ID,score> <RPARE> <SEMICOMMA>

2.1.1 词法记号、模式、词法单元（续）

- 历史上词法定义中的遗留问题：
  - 忽略空格带来的问题：（FORTRAN90）

```
DO 8 I = 3. 75
```

```
DO8I = 3.75
```

```
DO 8 I = 3, 75
```

```
DO 8 I = 3,75
```

解决办法：lookahead

  - 关键字是否保留

```
IF THEN THEN THEN=ELSE; ELSE …
```

2.1.1 词法记号、模式、词法单元（续）

- 关键字、保留字和标准标识符的区别：
  - 保留字：语言预先确定了含义的词法单元，程序员不可以对这样的词法单元重新声明含义。
  - 关键字：
    - 语言预先确定了含义的词法单元，
    - 有的语言不允许程序员重新声明含义，同保留字
    - 有的语言允许重新声明含义，视关键字出现的位置不同而确定是关键字还是重新声明的变量等。如Fortran，关键字不保留
  - 标准标识符：预先确定含义，但程序员可以重新声明，视声明的作用域确定其是声明的标识符还是标准标识符。

2.1.2 词法记号的属性

- 属性可用来区分同一类（记号名相同）的不同记号

  `position = initial + rate * 60`的记号名和属性值：

  ⟨**id**，指向符号表中`position`条目的指针⟩

  ⟨**assign _ op**⟩

  ⟨**id**，指向符号表中`initial`条目的指针⟩

  ⟨**add_op**⟩

  ⟨**id**，指向符号表中`rate`条目的指针⟩

  ⟨**mul_ op**⟩

  ⟨**num**，整数值60⟩

2.1.2 词法记号的属性（续）

- 一个特定记号可能有若干属性，这些属性可能在后续翻译中用到（例如，position是一个ID，他有单词名称、类型、作用域等属性）
- 但是在记号二元组中，属性值只有一个，它可以是一个指向属性记录（包含若干属性）的指针（例如position所在的符号表条目）。

2.1.3 词法错误

- 词法分析器对源程序采取非常局部的观点，因而难以发现如下错误：
  - `fi (a == f (x) )` …
- “紧急方式”（panic mode）的错误恢复：
  - 删除输入指针当前指向的若干个字符（剩余输入的前缀），直到词法分析器能够发现一个正确的记号为止。

2.1.3 词法错误（续）

- 另一种错误恢复方式是进行错误修补，即检查剩余输入的前缀能否用下面的一个变换变成一个合法的词法单元：
  - 删除一个多余字符。
  - 插入一个遗漏的字符。
  - 用一个正确的字符代替一个不正确的字符
  - 交换两个相邻字符。

2.2 词法记号的描述与识别

- 为了把单词（lexeme）识别为记号（token），一个重要的问题是：
  - 你如何知道哪几个字符组成的单词可识别为某类记号？
- 这就用到了“模式”（pattern），而正规式（regular expression，也叫正则表达式）就是一种形式化的“模式”。

2.2 词法记号的描述与识别

- 串和语言
- 正规式
- 正规定义
- 状态转换图

2.2.1 串和语言

- 字母表（alphabet）：符号的有限集合。例：$\Sigma = \{0, 1\}$，英文字母表，ASCII，Unicode
- 串（string）：符号的有穷序列。例：0110, ε（空串）
- 字母表上的串：该字母表符号的有穷序列。例：当字母表$\Sigma = \{0, 1\}$时，1001是字母表上的串，而020则不是。
- 串的长度：|s|=串中符号的个数，例：|000|=3。空串长度为0。
- 语言（language）：字母表上的一个串集。例：$\{\varepsilon, 0, 00, 000, \ldots\}$, $\{\varepsilon\}$, $\varnothing$，所有语法正确的C程序集合，所有语法正确的英语句子集合。
- 某语言的句子（sentence）或字（word）：属于该语言的串。

2.2.1 串和语言（续）

- 关于串的部分的术语：
  - 前缀（prefix）：从串s的尾部删去若干个（包括0个）符号后余下的部分称为s的前缀。
  - 后缀（suffix）：从串s的头部删去若干个（包括0个）符号后余下的部分称为s的后缀。
  - 子串（substring）：从串s删除任意前缀和后缀所得到的串称为s的子串。

2.2.1 串和语言（续）

- 串的运算：
  - 连接　xy，$s\varepsilon = \varepsilon s = s$
  - 积（指数、幂）　s^0为ε，s^i为$s^{i-1}s$（$i > 0$）
- 语言的运算：
  - 和（union）：$L \cup M = \{s \mid s \in L$ 或 $s \in M\}$
  - 连接（concatenation）：$LM = \{st \mid s \in L$ 且 $t \in M\}$
  - 指数：L^0是$\{\varepsilon\}$，L^i是$L^{i-1}L$（$i > 0$）
  - 闭包（Kleene closure）：$L^* = L^0 \cup L^1 \cup L^2 \cup \ldots$
  - 正闭包（Positive closure）：　$L^+ = L^1 \cup L^2 \cup \ldots$

2.2.1 串和语言（续）

- 例：

  若语言L,D的定义如下：

  L: { $A, B, \ldots, Z, a, b, \ldots, z$ }, D: { $0, 1, \ldots, 9$ }

  则下列操作可得到哪些新语言？

  $L \cup D, LD, L^6, L^*, L(L \cup D)^*, D^+$

- 从基本集合开始，利用语言上的运算可以定义新的语言。

2.2.2 正规式

- 正规式（regular expression，又称正则表达式）是一种特殊表达式，用于表示语言。通常它按照一组定义规则，由较简单的正规式构成。（模式）
- 正规式表示的语言称为正规集（regular set）。
- 我们约定正规式r表示的语言为$L(r)$。

2.2.2 正规式（续）

- 正规式的定义：
  - 归纳基础：ε， $a \in \Sigma$
  - 复合表达式：|， 连接，*（闭包）
  - 确定优先级：（ ）

2.2.2 正规式（续）

正规式	正规集	备注
ε	$\{\varepsilon\}$	
a	$\{a\}$	$a \in \Sigma$
(r)	$L(r)$	r是正规式
$(r) \mid (s)$	$L(r) \cup L(s)$	r和s是正规式
$(r)(s)$	$L(r)L(s)$	r和s是正规式
$(r)^*$	$(L(r))^*$	r是正规式

2.2.2 正规式（续）

- 若约定：
  - 闭包运算（*）优先级最高，且左结合；
  - 连接运算优先级次之，且左结合；
  - 或运算（|）优先级最低，且左结合，

则可省略不必要的括号，

如：$((a)\,((b)^*))|\,(c)$可以写成$ab^*|\,c$

2.2.2 正规式（续）

- 正规式的例子： $\Sigma = \{a, b\}$
  - $a \mid b$ $\{a, b\}$
  - $(a \mid b)(a \mid b)$ $\{aa, ab, ba, bb\}$
  - $aa \mid ab \mid ba \mid bb$ $\{aa, ab, ba, bb\}$
  - a^* 由字母a构成的所有串集
  - $(a \mid b)^*$ 由a和b构成的所有串集
- 复杂的例子

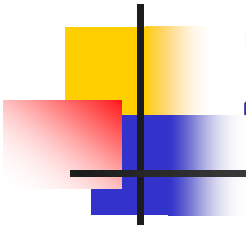
  (00 | 11 | ((01 | 10) (00 | 11) * (01 | 10))) *

  句子：01001101000010000010111001

2.2.2 正规式（续）

- 正规式的代数定律：可用于正规式的等价变换

$r\|s = s\|r$	|可交换
$r\|(s\|t) = (r\|s)\|t$	|可结合
$(rs)t = r(st)$	连接可结合
$r(s\|t) = rs\|rt;\ (s\|t)r = sr\|tr$	连接对|可分配
$\varepsilon r = r;\ r\varepsilon = r$	ε是连接的恒等元素
$r^* = (r\|\ \varepsilon)^* = \varepsilon\|rr^*$	ε肯定出现在一个闭包中
$r^{**} = r^*$	*是幂等的



2.2.2 正规式（续）

- 正规式不是串，也不是符号，它是一种模式，匹配该模式的所有串组成了一个语言（正规集）。

2.2.2 正规式（续）

- 练习：假设Σ={a, b}，请叙述下列正规式所描述的语言
  - (a|b)*a(a|b)
  - a*ba*ba*

  答案：
  - 以aa或ab结尾的a和b组成的串(倒数第二个符号是a的a和b组成的串)
  - 只含有2个b的a和b组成的串

2.2.2 正规式（续）

- 练习：请写出下列语言的正规式
  - 所有十进制偶数（注意：除0以外，首位不能为0，即04，00这种都不合法）。
  - 所有二进制偶数用正规式如何表示？

  答案：

((1|2|…|9)(0|1|…|9)*(0|2|4|6|8))|0|2|4|6|8

(0|1)*0

2.2.3 正规定义

- 正规定义（regular definition）就是为正规式命名，以求得表示上的简洁。
- 如果∑是基本符号的字母表，则正规定义是形式为

$$d_1 \rightarrow r_1$$
$$d_2 \rightarrow r_2$$
$$\cdots$$
$$d_n \rightarrow r_n$$

的定义序列。

各个d_i的名字都不同，且不属于∑

每个r_i都是$\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$上的正规式。

- 为区别名字和符号，下面用绿色表示名字。

2.2.3 正规定义（续）

- 正规定义的例子（例2.4）

C语言的标识符（字母、数字、下划线组成的串）集合：

$$\mathbf{letter_} \rightarrow \mathbf{A} \mid \mathbf{B} \mid \ldots \mid \mathbf{Z} \mid \mathbf{a} \mid \mathbf{b} \mid \ldots \mid \mathbf{z} \mid \mathbf{_}$$

$$\mathbf{digit} \rightarrow \mathbf{0} \mid \mathbf{1} \mid \ldots \mid \mathbf{9}$$

$$\mathbf{id} \rightarrow \mathbf{letter_}(\mathbf{letter_} \mid \mathbf{digit})^{*}$$

2.2.3 正规定义（续）

- 正规定义的例子（例2.5）

无符号数（整数、浮点数）集合，例
1946,11.28,63E8,1.99E–6

digit → 0 | 1 | … | 9
digits → digit digit*
optional_fraction → .digits|ε
optional_exponent → (E (+ | – | ε) digits) | ε
num → digits optional_fraction optional_exponent

2.2.3 正规定义（续）

- 正规式的扩展：（syntactic suger，语法糖。为了表示上的简洁）
  - 一个或多个实例：“+”，即a^+表示一个或多个a组成的所有串的集合。
    - 相关代数定律：$r^*=r^+|\varepsilon$ ； $r^+ = rr^*=r^*r$ 。
  - 零个或一个实例：“?”，即$r?$是$r|\varepsilon$的缩写。
  - “+”和“?”的优先级与结合性与“*”相同。
  - 字符组：$[abc]$表示正规式$a|b|c$，缩写字符组$[a\text{-}z]$表示$a|b|\ldots|z$ 等。$[e\text{-}gi\text{-}k]$表示什么？

    $e\,|\,f\,|\,g\,|\,i\,|\,j\,|\,k$

2.2.3 正规定义（续）

- 例2.5无符号数的简化表示：

  digit → [0-9]

  digits → digit$^+$

  num → digits (.digits)? (E[+-]? digits)?

- 例2.4标识符的简化表示：

  letter_ → [*A-Za-z*_]

  digit → [0-9]

  id → letter_(letter_|digit)*

2.2.4 状态转换图

- 前面关于正规式和正规定义的章节讨论怎样描述词法单元，而本节讨论怎样识别记号。
- 识别记号是由词法分析器进行的，而制作状态转换图是手工构造词法分析器的第一步。

2.2.4 状态转换图（续）

- 状态转换图（Transition Diagrams）描绘语法分析器为得到下一个记号而调用词法分析器时，词法分析器所做的动作。

2.2.4 状态转换图（续）

- 本小节的讨论基于这样一个正规定义 (例2.6)

while → **while**

do → **do**

relop → < | <= | = | <> | > | >=

letter → [*A-Za-z*]

digit → [0-9]

id → **letter** (**letter** | **digit**)*

num → **digit**+ (.**digit**+)? (E (+ | –)? **digit**+)?

delim → **blank** | **tab** | **newline**

ws → **delim**+

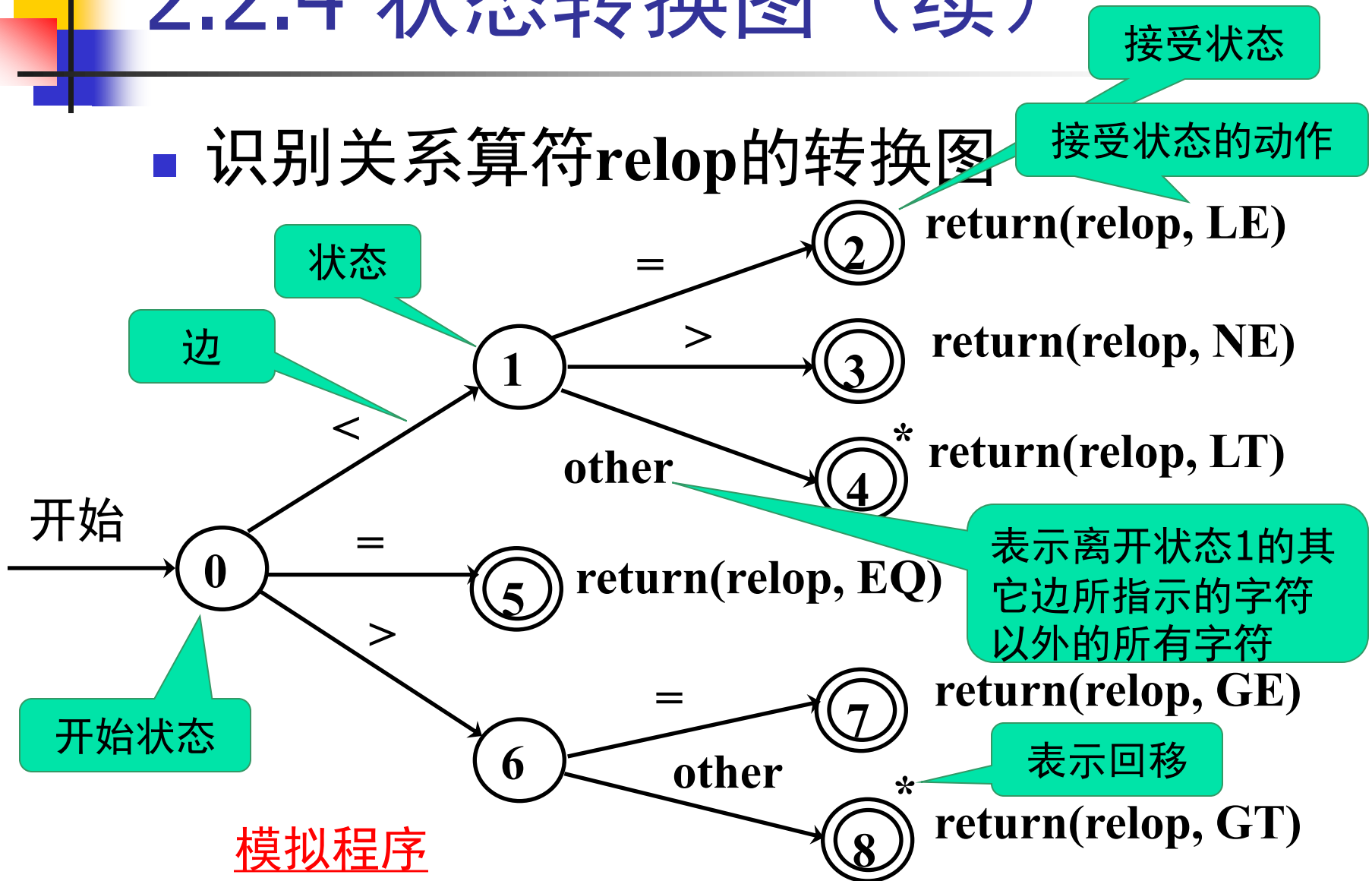
2.2.4 状态转换图（续）

- 正规式、记号名和属性的对应

正规式	记号名	属性值
ws	—	—
while	**while**	—
do	**do**	—
id	**id**	符号表条目的指针
num	**number**	数表条目的指针
relop	**relop**	LT、LE、EQ、NE、GT或GE

2.2.4 状态转换图（续）

- 识别关系算符relop的转换图



模拟程序

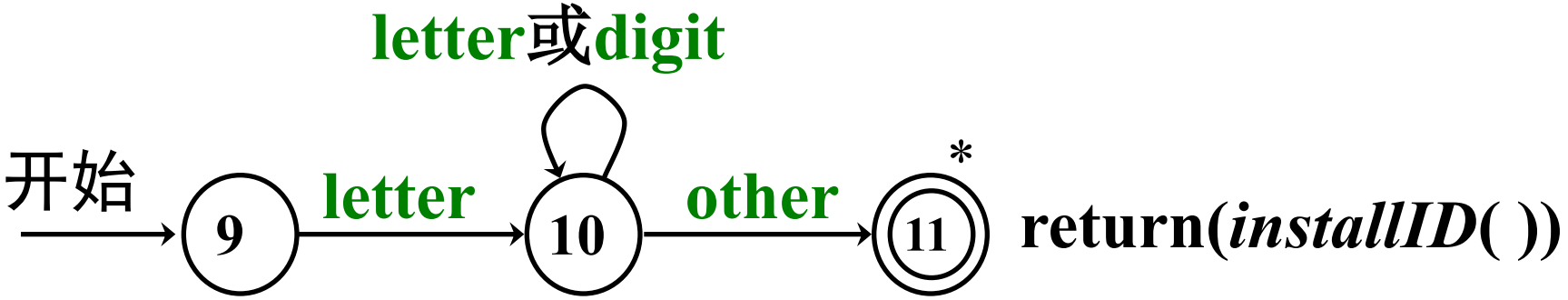
2.2.4 状态转换图（续）

- 识别标识符和关键字的转换图

  while → while

  do → do

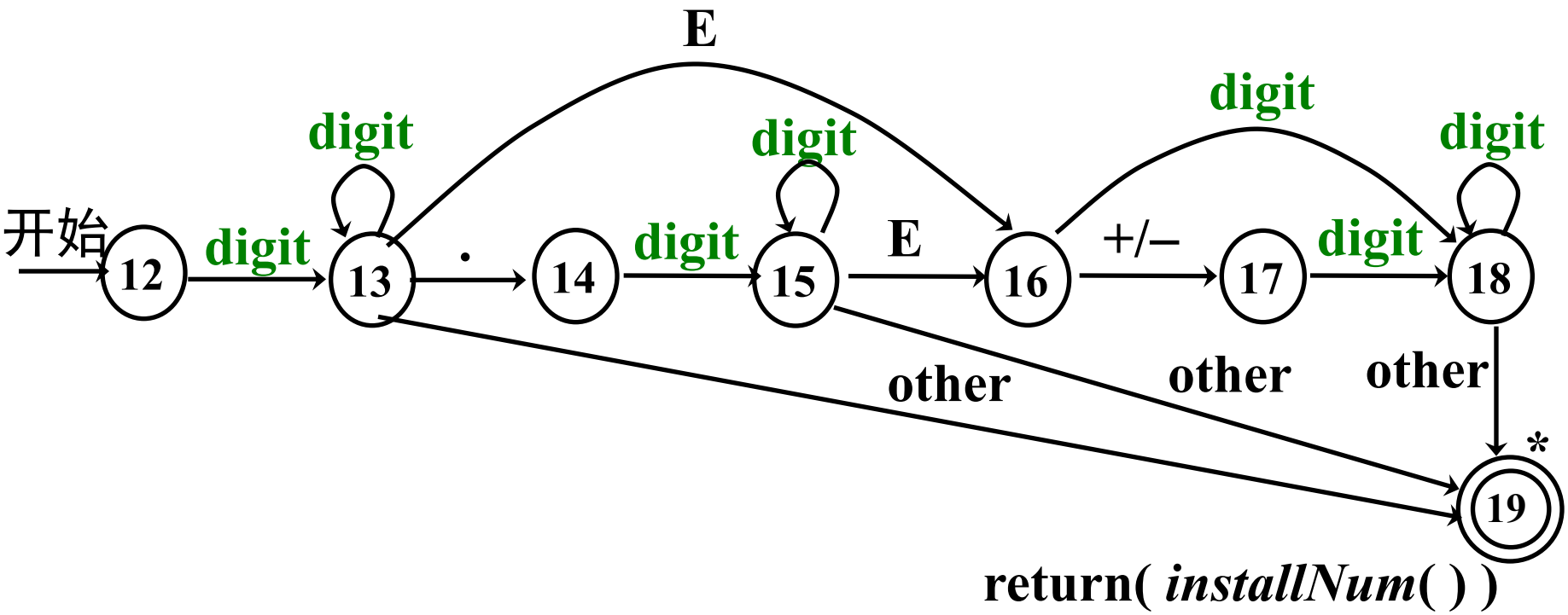
  id → letter (letter | digit)*



2.2.4 状态转换图（续）

- 识别无符号数的转换图

num → digit$^+$ (.digit$^+$)? (E (+ | –)? digit$^+$)?

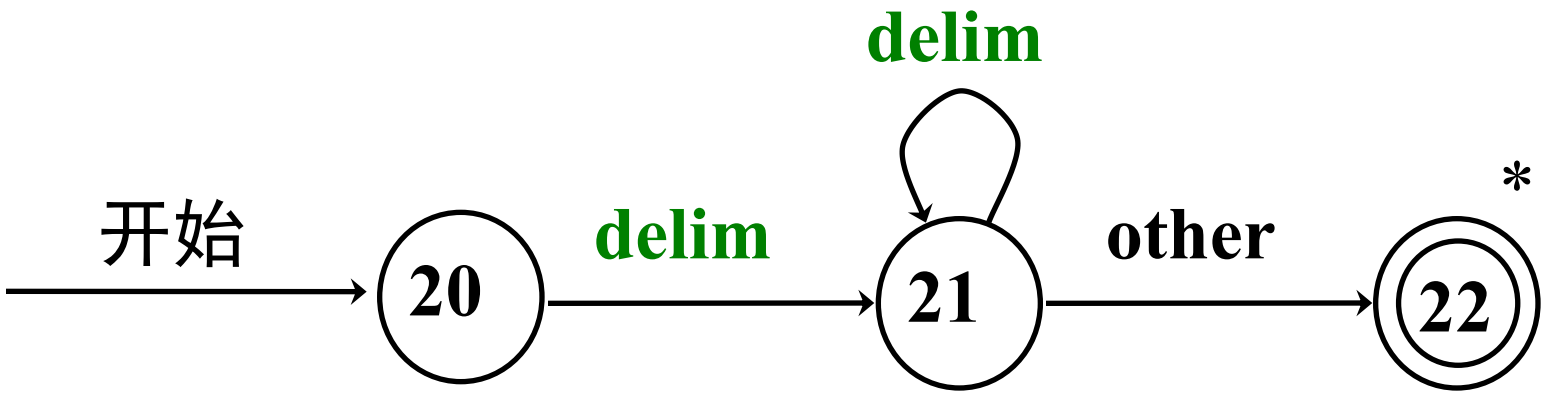


2.2.4 状态转换图（续）

- 识别空白的转换图

  delim → blank | tab | newline

  ws → delim^{+}



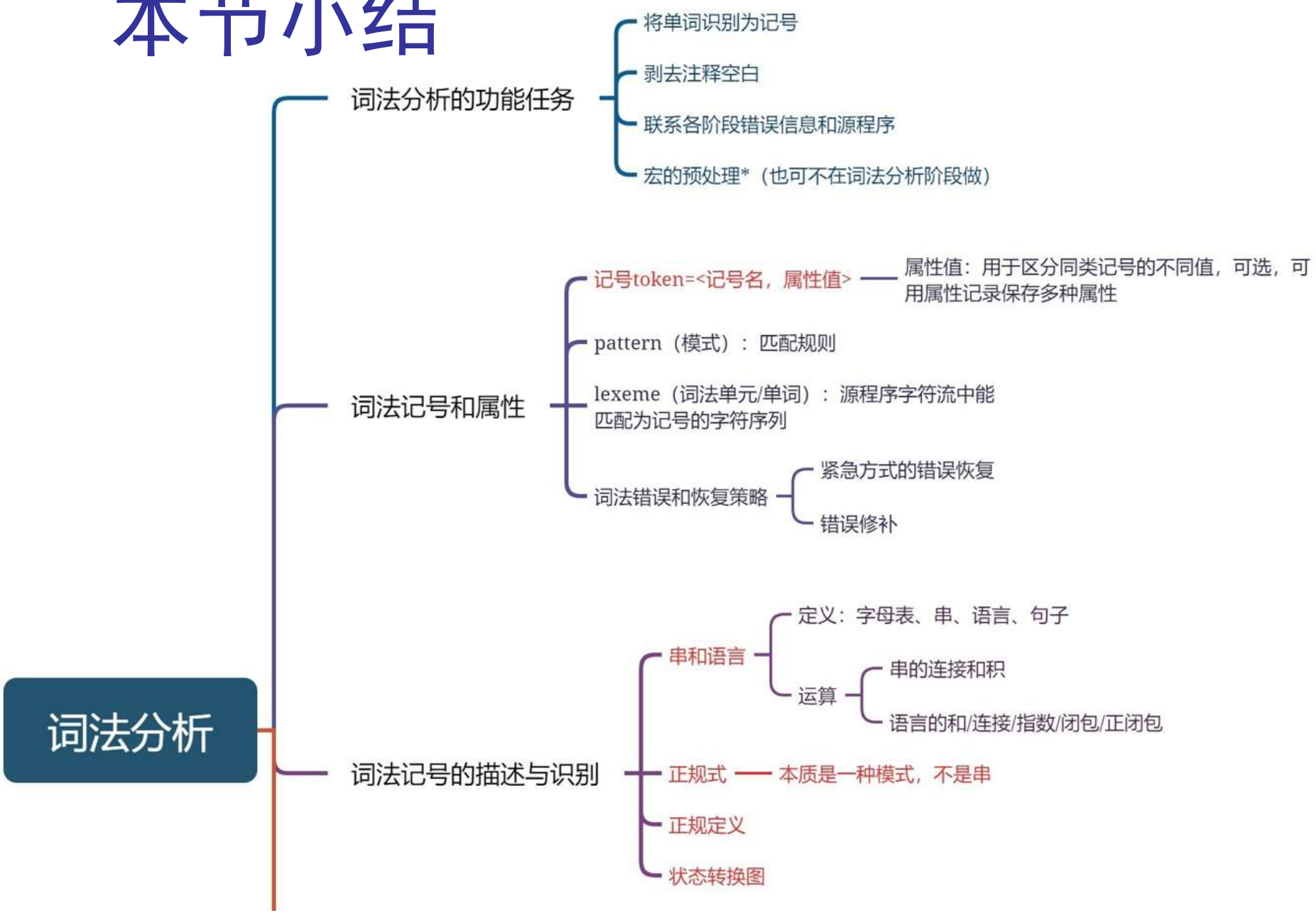
2.2.4 状态转换图（续）

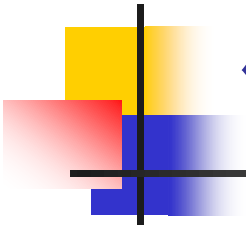
- 状态转换图的合并：
  - 前三个合并：将开始状态合并成一个。
  - 合并识别空白的转换图：
    - 开始状态合并
    - 识别空白的转换图的接受状态动作为：回到开始状态，识别下一个记号。

2.2.4 状态转换图（续）

- 构造词法分析器的一种简单办法：
  - 用状态转换图描述源语言的词法记号结构；
  - 手工把这种状态转换图翻译成识别词法记号的程序。
- 例如下面这段伪代码就模拟了上述识别relop的状态转换图的识别过程。

本节小结


词法分析
词法分析的功能任务
将单词识别为记号
剥去注释空白
联系各阶段错误信息和源程序
宏的预处理*（也可不在词法分析阶段做）
词法记号和属性
记号token=<记号名，属性值>
属性值：用于区分同类记号的不同值，可选，可用属性记录保存多种属性
pattern（模式）：匹配规则
lexeme（词法单元/单词）：源程序字符流中能匹配为记号的字符序列
词法错误和恢复策略
紧急方式的错误恢复
错误修补
词法记号的描述与识别
串和语言
定义：字母表、串、语言、句子
运算
串的连接和积
语言的和/连接/指数/闭包/正闭包
正规式
本质是一种模式，不是串
正规定义
状态转换图

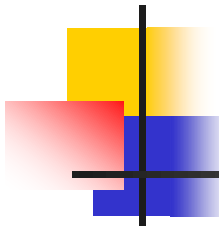


作业

- 本章重点是正规式。
- 作业要求：
  - 看到正规式能说出其表示的语言。
  - 对于一个非形式叙述的语言能写出正规式。

作业（续）

- P38/2.3
- P38/2.4 (f) (g)
- 分组作业：预习课本第二章2.3、2.4、2.5节，组内讨论，梳理主要内容，可以以思维导图的方式梳理，梳理的过程中有不懂的地方重点标出。将预习的成果整理为博客发表到雨课堂讨论区。（每组提交一份，下周二之前提交。命名：第N组1-2：预习。其中N为组号）



```
TOKEN getRelop(){
  TOKEN retToken = new(RELOP);
  while(1) { /* repeat character processing until
                a return or failure occurs */
    switch(state){
      case 0: c = nextChar();
              if (c == '<') state = 1;
              else if (c == '=') state = 5;
              else if (c == '>') state = 6;
              else fail (); /* lexeme is not a
                               relop */
              break;
      case 1: ...
        ...
      case 8: retract();
              retToken.attribute = GT;
              return(retToken) ;
    }
  }
}
```

状态转换图

返回